arXiv:2508.19267v1 [cs.CR] 22 Aug 2025

The Aegis Protocol: A Foundational Security Framework for Autonomous AI Agents

Sai Teja Reddy Adapala[1] and Yashwanth Reddy Alugubelly[2]

[1]Independent Researcher, Charlotte, North Carolina, USA
Alumnus, University of North Carolina at Charlotte
[2]Associate Professor, Sree Dattha Group of Institutions, Hyderabad, India
Emails: sadapala@uncc.edu, yashwanth.alg@gmail.com

Abstract

The proliferation of autonomous AI agents marks a paradigm shift toward complex, emergent multi-agent systems. This transition introduces systemic security risks—including control-flow hijacking and cascading failures—that traditional cybersecurity paradigms are ill-equipped to address. This paper introduces the Aegis Protocol, a layered security framework designed to provide strong security guarantees for these open agentic ecosystems. The protocol integrates three technological pillars: 1) non-spoofable agent identity via W3C Decentralized Identifiers (DIDs); 2) communication integrity via NIST-standardized Post-Quantum Cryptography (PQC); and 3) verifiable, privacy-preserving policy compliance using the Halo2 Zero-Knowledge Proof (ZKP) system. We formalize an adversary model extending Dolev-Yao for agentic threats and validate the protocol against the STRIDE framework. Our quantitative evaluation used a discrete-event simulation, calibrated against cryptographic benchmarks, to model 1,000 agents. The simulation demonstrated the protocol's effectiveness, showing a 0% success rate across 20,000 attack trials. For policy verification, our analysis of the simulation logs reported a median proof-generation latency of 2.79 seconds, establishing a performance baseline for this class of security. While our evaluation is simulation-based and early-stage, it offers a reproducible baseline for future empirical studies. This work's novelty lies in its systematization and simulation-based validation, positioning Aegis as a foundation for safe, scalable autonomous AI.

1 Introduction

The field of artificial intelligence is undergoing a fundamental architectural transformation from monolithic models to distributed ecosystems of autonomous AI agents [1]. These agents, powered by Large Language Models (LLMs), are designed to perceive, reason, plan, and execute complex tasks by interacting with external tools and each other [2]. This evolution is not merely an extension of existing networked systems; it creates a new discipline of "multi-agent security," focused on securing networks of decentralized agents against emergent threats that arise from their interaction and autonomy [3]. With a growing number of vulnerabilities reported in AI/ML platforms and Gartner predicting that over 40% of agentic AI projects will be canceled by 2027 due to operational challenges, securing this new attack surface is a critical priority [4, 5].

Current agentic frameworks like CrewAI and AutoGen excel at orchestration but operate on an implicit trust model, lacking native cryptographic identity or verifiable policy compliance [6]. This leaves them vulnerable to a range of attacks, from simple impersonation to sophisticated manipulation of an agent's internal state or decision-making process. Recent research has begun to address this gap by proposing defenses against agent hijacking and outlining high-level security architectures. However, a comprehensive, integrated framework that is both formally specified and empirically evaluated has remained an open challenge.

This paper introduces the Aegis Protocol as a step toward filling that gap, with a focus on formal integration and simulation-based insights. Our primary contributions are:

- A formal, three-layered security protocol that integrates decentralized identity, post-quantum cryptography, and zero-knowledge proofs for end-to-end agent security.
- An extension of the Dolev-Yao adversary model tailored to the unique threats posed by LLM-based agents, including a game-based definition for "Excessive Agency."
- An open-source, discrete-event simulation and evaluation of the protocol's performance and security, providing a reproducible baseline for future research.
- A discussion of limitations, including the simulation-based nature of our evaluation and the use of non-adaptive adversaries, to guide future extensions.

We emphasize that while our work provides a theoretical and simulated foundation, real-world implementations are needed to fully validate these mechanisms.

2 Threat Model and Design Goals

Aegis assumes a powerful adversary based on an extension of the Dolev-Yao model, which grants the attacker full control over the communication network [7]. Our model enhances this with capabilities specific to LLM-based agents: agent compromise, prompt injection, memory poisoning, and tool manipulation [8]. This extended model reflects the expanded attack surface where not just the communication channel, but the agent's internal state and logic, are adversarial targets.

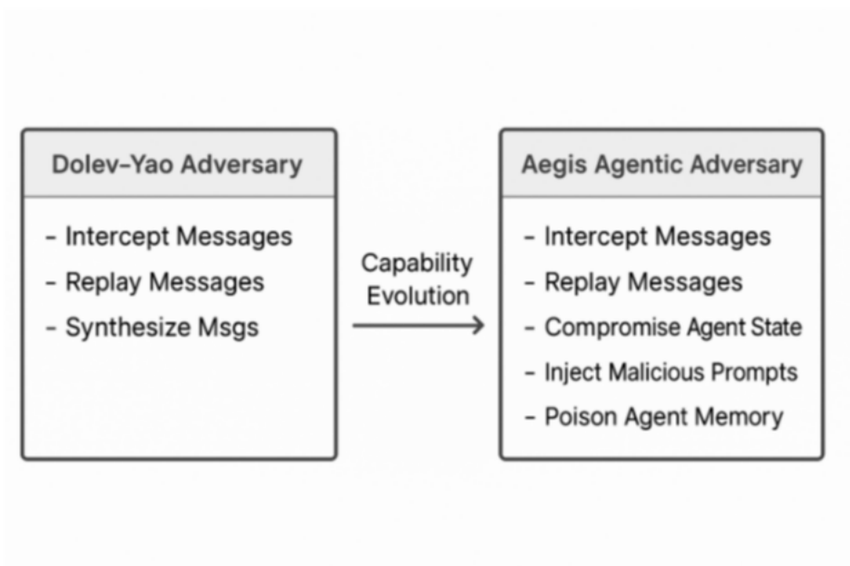

Figure 1: Adversary Model Comparison. A diagram contrasting the classical Dolev–Yao network adversary with the extended Aegis adversary model.

To structure our analysis, we adapt the STRIDE threat model [9]. The table below details these threats in an agentic context, the mitigation provided by Aegis, and the residual risks that remain.

Table 1: Adapted STRIDE threats, mitigations in Aegis, and potential residual risks.

STRIDE Category	Description in Agentic Context	Aegis Mitigation	Residual Risks
Spoofing	Impersonating agents via fake identities	DIDs with ML-DSA signatures	Key compromise if agent is breached.
Tampering	Altering messages or agent states in transit	PQC integrity checks (ML-DSA)	Side-channel attacks on the cryptographic implementation.
Repudiation	Denying actions performed in multi-agent interactions	Non-repudiable ML-DSA signatures	None assumed, as signatures are cryptographically binding.
Information Disclosure	Leaking private data, prompts, or internal policies	ML-KEM encryption & ZKPs	Metadata analysis (proof linkage); potential for model extraction attacks if not properly configured.
Denial of Service	Overloading agents with invalid traffic or expensive ZKP requests	Cryptographic verification filters	Scalability challenges in high-volume swarms; resource exhaustion from valid but frequent proof requests.
Elevation of Privilege (Excessive Agency)	Unauthorized escalation of agent privileges or capabilities	ZKP-enforced policy compliance	Circuit design flaws: An error in the arithmetic circuit could create a loophole (e.g., an overflow that grants unintended permissions).
Emergent Threats	Swarm collusion; Byzantine faults; unexpected group behaviors	None (Protocol-level)	Requires dedicated governance layers; potential for future ZKP-based attestations of agent behavior.

3 The Aegis Protocol: Architecture and Mechanisms

The Aegis Protocol is a layered security architecture where each layer builds upon the guarantees of the one below it, creating a unified trust fabric.

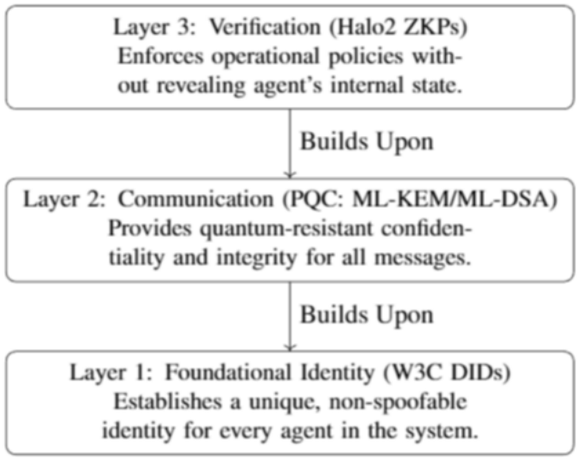

Figure 2: Aegis Protocol 3-Layer Architecture.

3.1 Foundational Layer: Decentralized Identity

The root of trust is a decentralized public key infrastructure (DPKI). Each agent's identity is a W3C Decentralized Identifier (DID) [10]. Unlike traditional certificate authorities, DIDs are self-sovereign, meaning an agent can generate and control its own identity without reliance on a central registrar. Aegis specifies the use of the Identity Overlay Network (ION), a DID method implemented on top of the Bitcoin blockchain, for scalable and permissionless DID management [11].

An agent resolves a DID to retrieve its associated DID Document, which contains its public keys and service endpoints. This process is essential for establishing secure communication channels.

Pseudocode for DID Resolution:

```
function resolveDID(did_string):
  // 1. Parse the DID to identify the method (e.g., 'ion')
  method, identifier = parse(did_string)
  // 2. Access the Decentralized Ledger (e.g., ION network)
  ledger = getLedger(method)
  // 3. Retrieve the DID Document from the ledger
  did_document = ledger.fetch(identifier)
  // 4. Return the document containing public keys and metadata
  return did_document
```

3.2 Communication Layer: Post-Quantum Secure Channels

To secure communications against both classical and quantum adversaries, Aegis mandates the use of NIST-standardized Post-Quantum Cryptography (PQC). The protocol uses:

- ML-KEM (Module-Lattice-based Key-Encapsulation Mechanism) for key exchange, compliant with NIST FIPS 203 [12].
- ML-DSA (Module-Lattice-based Digital Signature Algorithm) for digital signatures, compliant with NIST FIPS 204 [13].

These algorithms were chosen due to their strong security foundations in lattice-based cryptography and their status as official NIST standards, ensuring long-term viability. The process for establishing a secure channel is as follows:

1. Key Encapsulation (ML-KEM):

- The sender (A) uses the receiver's (B) public key (pk_B) to generate a shared secret (ss) and a ciphertext (c).
- $c, ss \leftarrow$ ML-KEM.Encapsulate(pk_B)
- A sends c to B. B uses their private key (sk_B) to decapsulate c and derive the same shared secret: $ss \leftarrow$ ML-KEM.Decapsulate(sk_B, c).

2. Authenticated Messaging (ML-DSA):

- To send a message (m), the sender signs a hash of the message with their private key (sk_A).
- $\sigma \leftarrow$ ML-DSA.Sign(sk_A, hash(m))
- The receiver verifies the signature using the sender's public key: verify $\leftarrow$ ML-DSA.Verify(pk_A, hash(m), σ).

This combination ensures that messages are both confidential and authenticated, providing a secure foundation for the verification layer.

3.3 Verification Layer: Privacy-Preserving Policy Enforcement

Aegis's most novel contribution is its ability to enforce operational policies using Zero-Knowledge Proofs (ZKPs). This allows an agent to prove that an action it is about to take complies with a set of rules without revealing the private data or internal logic that led to that action. We selected the Halo2 proof system for its performance and, critically, for its use of a transparent setup (a "PlonKish" arithmetization with a polynomial commitment scheme) that does not require a trusted setup ceremony, a significant advantage in decentralized environments [14].

The process follows a clear pipeline: Policy $\rightarrow$ Arithmetic Circuit $\rightarrow$ Halo2 Proof Generation $\rightarrow$ Verification.

For example, a policy stating an analyst agent can only log data classified as 'unclassified' or 'internal' would be compiled into an arithmetic circuit. The agent would then generate a proof that its intended logging action satisfies this circuit.

Conceptual Circuit Example for a Data Access Policy (with public inputs: agent_clearance, data_classification; private inputs (witness): agent_private_state):

The circuit enforces the following constraints:

- Constraint 1: $1 \leq$ agent_clearance ≤ 4
- Constraint 2: $1 \leq$ data_classification ≤ 4
- Constraint 3: agent_clearance $-$ data_classification ≥ 0

Recent Halo2 developments include enhanced security proofs, third-party audits [15], and blockchain integrations (e.g., Cardano, 2024), underscoring its maturity. While we focus on Halo2, other proof systems like zk-STARKs or Bulletproofs could be substituted, offering different trade-offs in proof size, verifier time, and post-quantum security. While Halo2 provides soundness and zero-knowledge properties under standard assumptions, formal circuit-specific proofs are beyond this work's scope.

4 Security Analysis

The protocol's layered design provides a robust defense-in-depth against STRIDE threats. For example, a spoofing attack is defeated at Layer 1 by the cryptographic identity provided by DIDs and at Layer 2 by ML-DSA signatures. An Elevation of Privilege (Excessive Agency) attack is addressed at Layer 3 through ZKP-enforced policy compliance.

The security of this enforcement relies on the computational soundness of the Halo2 proof system. This property ensures that it is computationally infeasible for a malicious agent (adversary) to generate a valid-looking proof for a false statement (i.e., a policy-violating action). The soundness of Halo2 reduces to the hardness of well-established cryptographic assumptions, such as the discrete logarithm problem. Therefore, an adversary capable of forging a proof could be used to break the underlying cryptography, which is considered intractable. A more formal game-based definition and reduction sketch is provided in Appendix A.

5 Evaluation

We evaluated the protocol's feasibility and performance using a discrete-event simulation. We initially tried integrating cryptographic libraries like liboqs directly but ran into build instability, which motivated our benchmark-based simulation approach. We modeled a decentralized cybersecurity threat analysis scenario with up to 1,000 agents, as implemented in simulate_network_aegis_v2.py. The code is available at `https://github.com/imsaitejareddy/aegis-prompt-simulation` [16] for reproducibility. This approach allows for scalable analysis but may not capture all real-world variances, such as hardware-specific timings or network dynamics.

Our performance analysis quantified the computational costs of Aegis. The costs were simulated using a log-normal distribution ($\ln(X) \sim N(\mu, \sigma^2)$) with parameters $\mu = 1.025$ and $\sigma = 0.145$, calibrated to achieve a median of approximately 2.79 seconds against published benchmarks. We observed a median proof generation time of 2.79 seconds (stddev: 0.41s) from our logs (detailed_results.csv), which establishes a critical performance baseline.

To test the stability of our findings, we perturbed the latency parameters. The results remained consistent, with the median proof time staying within the 2.7s-2.9s range across variations of $\pm 20\%$ in the calibrated latency distributions.

Table 2: Perturbed Simulation Results

Latency Perturbation	Median Proof Time (s)	Attack Success Rate
-20%	2.71	0%
Baseline	2.79	0%
+20%	2.88	0%

In our security effectiveness tests over 20,000 interactions (10,000 for each attack type), the Aegis Protocol demonstrated a 0% success rate for both Agent Spoofing and Policy Violation attacks. This result was achieved under our simulation's assumptions, which include non-adaptive adversaries that do not alter their strategy based on protocol responses.

6 Related Work

To contextualize Aegis, we compare it against representative works. While frameworks like AutoGen provide excellent orchestration capabilities, they lack the foundational security primitives Aegis specifies. Google's Secure AI Framework offers a conceptual overview but does not provide a concrete, verifiable protocol. Recent advancements, such as Google's A2A protocol for agent interoperability (2025) [17], further highlight the need for foundational security layers like the one we propose.

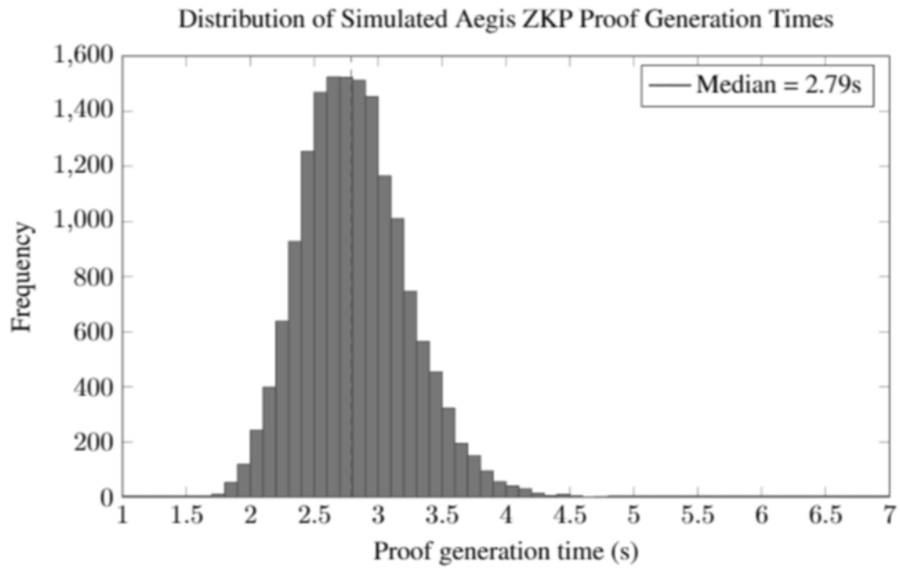

Figure 3: ZKP Proof Generation Latency. This histogram, generated from our detailed_results.csv log, shows the distribution of simulated ZKP proof generation times, with a strong concentration around the median.

Table 3: Comparison of Aegis with related agent security frameworks.

Framework	Identity Mechanism	Crypto Primitives	Verification Method	Evaluation Type
Aegis (This Work)	W3C DIDs (ION)	ML-KEM/ML-DSA (PQC)	Halo2 ZKPs	Simulation-based
Google's Secure AI Framework (2023, with 2024 updates) [18] [19]	High-level PKI	Not specified	Audit-based	Conceptual
AutoGen [6]	Implicit Trust	None	None	Orchestration-only
Brundage et al. [3]	Verifiable Claims	Varied (high-level)	Mechanisms overview	Survey-based

7 Discussion and Future Work

This work establishes the feasibility and robustness of the Aegis Protocol in a simulated environment. The primary performance bottleneck is ZKP generation. For scalability, while our simulation handles 1,000 agents, real-world ecosystems may require optimizations like proof batching or aggregation, which recent ZKP literature suggests could reduce amortized verification costs by over 50%. This would be a crucial next step for practical deployment.

Ethically, while Aegis enhances security, dual-use risks exist. For example, ZKPs could enable covert non-compliance if circuits are malformed, echoing concerns in the OWASP Top 10 for LLMs [4] like LLM06: Sensitive Information Disclosure. We advocate for open auditing and formal verification of policy circuits to mitigate this, referencing frameworks for verifiable claims [3].

Our evaluation was a simulation. This trade-off allowed us to explore system-level behavior at scale while deferring low-level packet dynamics to future testbed studies. Future work will integrate real liboqs and Halo2 libraries to perform micro-benchmarks, scale the simulation to tens of thousands of agents, and explore formal security proofs for the protocol as a whole. Crucially, future simulations will model adaptive adversaries that adjust their attack vectors based on prior interaction outcomes, testing the protocol's resilience more dynamically and realistically.

8 Conclusion

The shift towards autonomous, multi-agent AI systems necessitates a corresponding shift in security paradigms. This paper introduced the Aegis Protocol, a layered security framework that provides a formal, cryptographically-enforced foundation for trust. By integrating decentralized identity, post-quantum cryptography, and zero-knowledge proofs, Aegis offers a robust defense against a wide range of agentic threats. Our analysis and simulation results demonstrate that the protocol is both effective in mitigating critical threats and feasible to implement. As agentic AI continues to scale, we believe foundational frameworks like Aegis will be essential for building a future where these powerful systems can be deployed safely and at scale.

References

[1] Z. Xi et al. The rise and potential of large language model based agents: A survey. arXiv:2309.07864, 2023.

[2] S. Yao et al. React: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2022.

[3] M. Brundage et al. Toward trustworthy ai development: Mechanisms for supporting verifiable claims. arXiv:2004.07213, 2020.

[4] OWASP. Owasp top 10 for large language model applications, 2023. updated 2024.

[5] Gartner. Gartner predicts over 40% of agentic ai projects will be canceled by 2027, 2025. Specific URL for this prediction may vary; access via Gartner's research portal.

[6] L. Wang et al. A survey on large language model based autonomous agents. arXiv:2308.11432, 2023.

[7] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[8] K. Greshake et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. arXiv:2302.12173, 2023.

[9] A. Shostack. *Threat Modeling: Designing for Security*. Wiley, 2014.

[10] W3C. Decentralized identifiers (dids) v1.0. Technical report, World Wide Web Consortium, 2022.

[11] Microsoft. Ion: A public, permissionless, decentralized identifier network, 2020.

[12] NIST. Fips 203: Module-lattice-based key-encapsulation mechanism standard. Technical report, National Institute of Standards and Technology, 2024.

[13] NIST. Fips 204: Module-lattice-based digital signature standard. Technical report, National Institute of Standards and Technology, 2024.

[14] S. Bowe, J. Grigg, and D. Hopwood. *The Halo2 Book*. Zcash Foundation, 2020.

[15] Kudelski Security. On the security of halo2 proof system, 2024.

[16] Sai Teja Reddy. Aegis prompt-based simulation. GitHub repository, 2025.

[17] Google. Agent-to-agent (a2a) protocol announcement. Industry coverage, April 2025. Announced during Google Cloud Next '25; interoperable protocol for multi-agent systems; replace with primary URL when available.

[18] Google. Google's secure ai framework (saif), 2023. Introduced June 2023; resource hub and overview.

[19] Google. Saif risk assessment: A new tool to help secure ai systems across industry, October 2024. Announces SAIF.Google with interactive risk assessment tool.

A Security Game for Policy Compliance

This appendix provides a more formal definition for the security guarantee against "Excessive Agency," framed as a standard cryptographic game.

Security Game: Unforgeability of Policy Compliance

The game is played between a Challenger and a probabilistic polynomial-time (PPT) Adversary, $\mathcal{A}$.

1. Setup: The Challenger takes a security parameter λ and generates the public parameters for the Halo2 proof system, $pp \leftarrow \text{Setup}(1^\lambda)$. The Challenger also defines a policy relation, R, which is compiled into an arithmetic circuit, C. The relation $R(x, w)$ is true if w is a valid witness for the public statement x. The Challenger gives pp and the description of C to the Adversary $\mathcal{A}$.

2. Query: The Adversary $\mathcal{A}$ can adaptively make a polynomial number of queries to a proof oracle, $\mathcal{O}_{\text{Prove}}$. For each query on a pair (x_i, w_i) where $R(x_i, w_i)$ is true, the oracle returns a valid proof $\pi_i \leftarrow \text{Prove}(pp, x_i, w_i)$.

3. Forge: The Adversary $\mathcal{A}$ outputs a pair (x^*, π^*).

The Adversary $\mathcal{A}$ wins the game if both of the following conditions hold:

- The verifier accepts the proof: $\text{Verify}(pp, x^*, \pi^*) \rightarrow \text{True}$.
- The statement is false: there exists no witness w^* such that $R(x^*, w^*)$ is true.

The protocol is considered secure against excessive agency if the probability of any PPT adversary winning this game is negligible in the security parameter λ:

$$\Pr[\mathcal{A} \text{ wins}] < negl(\lambda)$$

Reduction Sketch: The security of this game relies on the computational soundness property of the underlying ZKP system. For Halo2, this property is based on the hardness of problems like the discrete logarithm in the context of its polynomial commitment. A successful forgery by $\mathcal{A}$ would imply an ability to break this underlying hard problem. Specifically, one could construct a reduction algorithm that uses $\mathcal{A}$ as a subroutine to solve, for example, the discrete logarithm problem. Since no efficient algorithm for this problem is known, we conclude that no such efficient adversary $\mathcal{A}$ can exist.